Report Of Mini Project

Title Of Project: Student Record Management

System

Name Of Student: Bhakti Bibhishan Bhoskar

Roll No:13125

- **AIM:**

The aim of the student record management system is to develop a computerized platform to efficiently store, manage, and retrieve student information.
It helps reduce manual work, ensure data accuracy, and provide quick access to student records.

- **USE CASE:**

- **Add Student Records:** Administrator can add new student details such as name, roll number, and course information.
- **Update Information:** Existing student data can be edited or updated easily when changes occur.
- **View Records:** Teachers or administrators can quickly retrieve and view student profiles and academic details.
- **Delete Records:** Inactive or duplicate student entries can be securely removed from the database.

- **MYSQL TABLES:**

C:\Users\Admin>mysql -u root -p
Enter password: **********
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 14
Server version: 8.0.44 MySQL Community Server - GPL

Copyright (c) 2000, 2025, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql> CREATE DATABASE rec;
Query OK, 1 row affected (0.01 sec)

mysql> use rec;
Database changed
mysql> CREATE TABLE student (rollNo INT PRIMARY KEY,name
VARCHAR(100),fname VARCHAR(100),sub VARCHAR(100),grade
VARCHAR(10));
Query OK, 0 rows affected (0.07 sec)

mysql> select * from student;
Empty set (0.00 sec)

mysql> select * from student;
+--------+--------+---------+------+-------+
| rollNo | name   | fname   | sub  | grade |
+--------+--------+---------+------+-------+
|     22 | neha   | kohle   | java | 8.7   |
|     25 | bhakti | bhoskar | DBMS | 9.1   |
+--------+--------+---------+------+-------+
2 rows in set (0.00 sec)

mysql> select * from student;
+--------+----------+---------+--------+-------+
| rollNo | name     | fname   | sub    | grade |
+--------+----------+---------+--------+-------+
|      1 | piya     | patil   | TOC    | 8.6   |
|     22 | neha     | kohle   | java   | 8.7   |
|     25 | bhakti   | bhoskar | DBMS   | 9.1   |
|     45 | shravani | kavle   | python | 9.8   |
+--------+----------+---------+--------+-------+
4 rows in set (0.00 sec)
mysql> select * from student;
+--------+----------+---------+--------+-------+
| rollNo | name     | fname   | sub    | grade |
+--------+----------+---------+--------+-------+
|      1 | piya     | patil   | SPOS   | 8.6   |
|     22 | neha     | kohle   | java   | 8.7   |
|     25 | bhakti   | bhoskar | DBMS   | 9.1   |
|     45 | shravani | kavle   | python | 9.8   |
```

4 rows in set (0.00 sec)

```
mysql> select * from student;
+--------+----------+---------+--------+-------+
| rollNo | name     | fname   | sub    | grade |
+--------+----------+---------+--------+-------+
|     22 | neha     | kohle   | java   | 8.7   |
|     25 | bhakti   | bhoskar | DBMS   | 9.1   |
|     45 | shravani | kavle   | python | 9.8   |
+--------+----------+---------+--------+-------+
3 rows in set (0.00 sec)

mysql>
```

- **SOURCE CODE IN PYTHON/FRONT END CODE:**

```
import tkinter as tk
from tkinter import ttk, messagebox
import pymysql

class StudentApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Student Record Management System")
        self.width = self.root.winfo_screenwidth()
        self.height = self.root.winfo_screenheight()
        self.root.geometry(f"{self.width}x{self.height}+0+0")
        self.root.configure(bg="#eef2f3")

        # -------- Title Section -------- #
        title = tk.Label(
            self.root,
            text=" 🎓 Student Record Management System",
            bd=5,
            relief="flat",
            bg="#43cea2",
            fg="white",
            font=("Segoe UI Black", 40, "bold")
        )
        title.pack(side="top", fill="x")

        # -------- Option Frame -------- #
```

```
optFrame = tk.Frame(self.root, bd=0, relief="flat", bg="#edf2fb")
optFrame.place(width=self.width / 3.2, height=self.height - 180, x=50, y=120)

options = [
    ("Add Student", self.addFrameFun),
    ("Search Student", self.searchFrameFun),
    ("Update Record", self.updFrameFun),
    ("Show All", self.showAll),
    ("Remove Student", self.delFrameFun)
]

for i, (text, command) in enumerate(options):
    tk.Button(
        optFrame,
        text=text,
        command=command,
        font=("Segoe UI Semibold", 18),
        bg="#48cae4",
        fg="white",
        activebackground="#00b4d8",
        activeforeground="white",
        bd=0,
        relief="flat",
        width=18,
        height=1,
        cursor="hand2"
    ).grid(row=i, column=0, pady=25, padx=50)

# -------- Detail Frame -------- #
```

```
        self.detFrame = tk.Frame(self.root, bd=3, relief="groove", bg="#d9f8c4")
        self.detFrame.place(width=self.width / 2 + 100, height=self.height - 180,
x=self.width / 3 + 80, y=120)

        lbl = tk.Label(self.detFrame, text="📋 Student Details", font=("Segoe UI
Black", 26),
                       bg="#d9f8c4", fg="#023047")
        lbl.pack(side="top", fill="x", pady=10)

        self.tabFun()

    # -------- Table Frame -------- #
    def tabFun(self):
        tabFrame = tk.Frame(self.detFrame, bd=3, relief="ridge", bg="white")
        tabFrame.place(width=self.width / 2, height=self.height - 300, x=35, y=80)

        x_scroll = tk.Scrollbar(tabFrame, orient="horizontal")
        y_scroll = tk.Scrollbar(tabFrame, orient="vertical")
        x_scroll.pack(side="bottom", fill="x")
        y_scroll.pack(side="right", fill="y")

        style = ttk.Style()
        style.configure("Treeview.Heading", font=("Segoe UI Semibold", 13),
background="#48cae4", foreground="black")
        style.configure("Treeview", font=("Segoe UI", 12), rowheight=28,
background="#f9f9f9", fieldbackground="#f9f9f9")

        self.table = ttk.Treeview(
            tabFrame,
```

```
            columns=("roll", "name", "fname", "sub", "grade"),
            xscrollcommand=x_scroll.set,
            yscrollcommand=y_scroll.set
        )
        x_scroll.config(command=self.table.xview)
        y_scroll.config(command=self.table.yview)

        self.table.heading("roll", text="Roll No")
        self.table.heading("name", text="Name")
        self.table.heading("fname", text="Last Name")
        self.table.heading("sub", text="Subject")
        self.table.heading("grade", text="Grade")

        self.table["show"] = "headings"
        for col in ("roll", "name", "fname", "sub", "grade"):
            self.table.column(col, width=150, anchor="center")

        self.table.pack(fill="both", expand=True)

    # -------- Add Student -------- #
    def addFrameFun(self):
        self.newWindow("Add Student", [
            ("Roll No", "roll"),
            ("Name", "name"),
            ("Last Name", "fname"),
            ("Subject", "sub"),
            ("Grade", "grade")
        ], self.addFun)
```

```
    def addFun(self):
        try:
            rn = self.entries["roll"].get().strip()
            name = self.entries["name"].get().strip()
            fname = self.entries["fname"].get().strip()
            sub = self.entries["sub"].get().strip()
            grade = self.entries["grade"].get().strip()

            if not all([rn, name, fname, sub, grade]):
                messagebox.showerror("Error", "Please fill all input fields!")
                return

            self.dbFun()
            query = "INSERT INTO student (rollNo, name, fname, sub, grade) VALUES
(%s, %s, %s, %s, %s)"
            self.cur.execute(query, (rn, name, fname, sub, grade))
            self.con.commit()
            messagebox.showinfo("Success", f"Student {name} (Roll No: {rn}) added
successfully!")
            self.showAll()
            self.popup.destroy()
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass
```

```
    # -------- Search Student -------- #
    def searchFrameFun(self):
        # note: combo=True means create combobox for 'option' AND a value entry for 'value'
        self.newWindow("Search Student", [("Search By", "option"), ("Value", "value")], self.searchFun, combo=True)

    def searchFun(self):
        # Validate selection and perform search
        opt = self.option.get().strip()
        val = self.value.get().strip()

        # Allowed columns
        allowed = ("rollNo", "name", "sub")
        if opt not in allowed:
            messagebox.showerror("Error", "Please select a valid field (rollNo, name or sub).")
            return
        if not val:
            messagebox.showerror("Error", "Please enter a value to search.")
            return

        try:
            self.dbFun()
            query = f"SELECT * FROM student WHERE {opt}=%s"
            self.cur.execute(query, (val,))
            data = self.cur.fetchall()
            self.table.delete(*self.table.get_children())
            for i in data:
```

```
            self.table.insert('', tk.END, values=i)
        # close connection & popup
        self.popup.destroy()
    except Exception as e:
        messagebox.showerror("Error", f"Error: {e}")
    finally:
        try:
            self.con.close()
        except:
            pass

# -------- Update Record -------- #
def updFrameFun(self):
    self.newWindow("Update Record", [
        ("Field (name/sub/grade)", "option"),
        ("New Value", "value"),
        ("Roll No", "roll")
    ], self.updFun)

def updFun(self):
    opt = self.entries["option"].get().strip()
    val = self.entries["value"].get().strip()
    rNo = self.entries["roll"].get().strip()

    if opt not in ("name", "sub", "grade"):
        messagebox.showerror("Error", "Field must be one of name, sub or grade.")
        return
    if not (val and rNo):
        messagebox.showerror("Error", "Please provide both new value and roll
```

```
number.")
            return

        try:
            self.dbFun()
            query = f"UPDATE student SET {opt}=%s WHERE rollNo=%s"
            self.cur.execute(query, (val, rNo))
            self.con.commit()
            messagebox.showinfo("Success", f"Record Updated for Roll No {rNo}")
            self.showAll()
            self.popup.destroy()
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass

    # -------- Delete Record -------- #
    def delFrameFun(self):
        self.newWindow("Remove Student", [("Roll No", "roll")], self.delFun)

    def delFun(self):
        rNo = self.entries["roll"].get().strip()
        if not rNo:
            messagebox.showerror("Error", "Please enter Roll No.")
            return
        try:
```

```
            self.dbFun()
            self.cur.execute("DELETE FROM student WHERE rollNo=%s", (rNo,))
            self.con.commit()
            messagebox.showinfo("Success", f"Student Roll No {rNo} removed
successfully.")
            self.showAll()
            self.popup.destroy()
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass

    # -------- Database Connection -------- #
    def dbFun(self):
        # change 'yourpassword' to your actual DB password
        self.con = pymysql.connect(host="localhost", user="root",
passwd="Shra@2807", database="rec")
        self.cur = self.con.cursor()

    # -------- Reusable Small Window -------- #
    def newWindow(self, title, fields, command, combo=False):
        # Destroys any previous popup reference (safe)
        if hasattr(self, "popup") and self.popup.winfo_exists():
            try:
                self.popup.destroy()
            except:
```

```
            pass

        self.popup = tk.Toplevel(self.root)
        self.popup.title(title)
        self.popup.geometry("460x420+400+200")
        self.popup.configure(bg="#caf0f8")
        self.entries = {}

        tk.Label(self.popup, text=title, font=("Segoe UI Black", 20), bg="#00b4d8",
fg="white", relief="flat").pack(fill="x", pady=5)

        frame = tk.Frame(self.popup, bg="#caf0f8")
        frame.pack(pady=20)

        for i, (label, key) in enumerate(fields):
            tk.Label(frame, text=f"{label}:", font=("Segoe UI Semibold", 13),
bg="#caf0f8").grid(row=i, column=0, pady=10, padx=10)
            if combo and key == "option":
                # create combobox for selecting column
                self.option = ttk.Combobox(frame, values=("rollNo", "name", "sub"),
font=("Segoe UI", 12), state="readonly")
                self.option.set("Select Field")
                self.option.grid(row=i, column=1, padx=10, pady=10)
            elif combo and key == "value":
                # for combo mode we also need a separate entry for the value
                self.value = tk.Entry(frame, font=("Segoe UI", 12), width=20, bd=2,
relief="groove")
                self.value.grid(row=i, column=1, padx=10, pady=10)
            else:
```

```
            entry = tk.Entry(frame, font=("Segoe UI", 12), width=20, bd=2,
relief="groove")
            entry.grid(row=i, column=1, padx=10, pady=10)
            # store in entries dict for normal (non-combo) or update windows
            self.entries[key] = entry

        tk.Button(self.popup, text="Submit", command=command, bg="#00b4d8",
fg="white",
                  font=("Segoe UI", 14, "bold"), bd=0, relief="flat", width=15,
cursor="hand2").pack(pady=15)

    def showAll(self):
        try:
            self.dbFun()
            self.cur.execute("SELECT * FROM student")
            data = self.cur.fetchall()
            self.table.delete(*self.table.get_children())
            for i in data:
                self.table.insert('', tk.END, values=i)
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass

root = tk.Tk()
obj = StudentApp(root)
```

```
root.mainloop()
import tkinter as tk
from tkinter import ttk, messagebox
import pymysql

class StudentApp:
    def __init__(self, root):
        self.root = root
        self.root.title("Student Record Management System")
        self.width = self.root.winfo_screenwidth()
        self.height = self.root.winfo_screenheight()
        self.root.geometry(f"{self.width}x{self.height}+0+0")
        self.root.configure(bg="#eef2f3")

        # -------- Title Section -------- #
        title = tk.Label(
            self.root,
            text=" 🎓 Student Record Management System",
            bd=5,
            relief="flat",
            bg="#43cea2",
            fg="white",
            font=("Segoe UI Black", 40, "bold")
        )
        title.pack(side="top", fill="x")

        # -------- Option Frame -------- #
        optFrame = tk.Frame(self.root, bd=0, relief="flat", bg="#edf2fb")
```

```
optFrame.place(width=self.width / 3.2, height=self.height - 180, x=50, y=120)

options = [
    ("Add Student", self.addFrameFun),
    ("Search Student", self.searchFrameFun),
    ("Update Record", self.updFrameFun),
    ("Show All", self.showAll),
    ("Remove Student", self.delFrameFun)
]

for i, (text, command) in enumerate(options):
    tk.Button(
        optFrame,
        text=text,
        command=command,
        font=("Segoe UI Semibold", 18),
        bg="#48cae4",
        fg="white",
        activebackground="#00b4d8",
        activeforeground="white",
        bd=0,
        relief="flat",
        width=18,
        height=1,
        cursor="hand2"
    ).grid(row=i, column=0, pady=25, padx=50)

# -------- Detail Frame -------- #
self.detFrame = tk.Frame(self.root, bd=3, relief="groove", bg="#d9f8c4")
```

```
        self.detFrame.place(width=self.width / 2 + 100, height=self.height - 180,
x=self.width / 3 + 80, y=120)

        lbl = tk.Label(self.detFrame, text="  Student Details", font=("Segoe UI
Black", 26),
                       bg="#d9f8c4", fg="#023047")
        lbl.pack(side="top", fill="x", pady=10)

        self.tabFun()

    # -------- Table Frame -------- #
    def tabFun(self):
        tabFrame = tk.Frame(self.detFrame, bd=3, relief="ridge", bg="white")
        tabFrame.place(width=self.width / 2, height=self.height - 300, x=35, y=80)

        x_scroll = tk.Scrollbar(tabFrame, orient="horizontal")
        y_scroll = tk.Scrollbar(tabFrame, orient="vertical")
        x_scroll.pack(side="bottom", fill="x")
        y_scroll.pack(side="right", fill="y")

        style = ttk.Style()
        style.configure("Treeview.Heading", font=("Segoe UI Semibold", 13),
background="#48cae4", foreground="black")
        style.configure("Treeview", font=("Segoe UI", 12), rowheight=28,
background="#f9f9f9", fieldbackground="#f9f9f9")

        self.table = ttk.Treeview(
            tabFrame,
            columns=("roll", "name", "fname", "sub", "grade"),
```

```
            xscrollcommand=x_scroll.set,
            yscrollcommand=y_scroll.set
        )
        x_scroll.config(command=self.table.xview)
        y_scroll.config(command=self.table.yview)

        self.table.heading("roll", text="Roll No")
        self.table.heading("name", text="Name")
        self.table.heading("fname", text="Last Name")
        self.table.heading("sub", text="Subject")
        self.table.heading("grade", text="Grade")

        self.table["show"] = "headings"
        for col in ("roll", "name", "fname", "sub", "grade"):
            self.table.column(col, width=150, anchor="center")

        self.table.pack(fill="both", expand=True)

    # -------- Add Student -------- #
    def addFrameFun(self):
        self.newWindow("Add Student", [
            ("Roll No", "roll"),
            ("Name", "name"),
            ("Last Name", "fname"),
            ("Subject", "sub"),
            ("Grade", "grade")
        ], self.addFun)

    def addFun(self):
```

```
        try:
            rn = self.entries["roll"].get().strip()
            name = self.entries["name"].get().strip()
            fname = self.entries["fname"].get().strip()
            sub = self.entries["sub"].get().strip()
            grade = self.entries["grade"].get().strip()

            if not all([rn, name, fname, sub, grade]):
                messagebox.showerror("Error", "Please fill all input fields!")
                return

            self.dbFun()
            query = "INSERT INTO student (rollNo, name, fname, sub, grade) VALUES
(%s, %s, %s, %s, %s)"
            self.cur.execute(query, (rn, name, fname, sub, grade))
            self.con.commit()
            messagebox.showinfo("Success", f"Student {name} (Roll No: {rn}) added
successfully!")
            self.showAll()
            self.popup.destroy()
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass

    # -------- Search Student -------- #
```

```
    def searchFrameFun(self):
        # note: combo=True means create combobox for 'option' AND a value entry for
'value'
        self.newWindow("Search Student", [("Search By", "option"), ("Value",
"value")], self.searchFun, combo=True)

    def searchFun(self):
        # Validate selection and perform search
        opt = self.option.get().strip()
        val = self.value.get().strip()

        # Allowed columns
        allowed = ("rollNo", "name", "sub")
        if opt not in allowed:
            messagebox.showerror("Error", "Please select a valid field (rollNo, name or
sub).")
            return
        if not val:
            messagebox.showerror("Error", "Please enter a value to search.")
            return

        try:
            self.dbFun()
            query = f"SELECT * FROM student WHERE {opt}=%s"
            self.cur.execute(query, (val,))
            data = self.cur.fetchall()
            self.table.delete(*self.table.get_children())
            for i in data:
                self.table.insert('', tk.END, values=i)
```

```
            # close connection & popup
            self.popup.destroy()
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass

    # -------- Update Record -------- #
    def updFrameFun(self):
        self.newWindow("Update Record", [
            ("Field (name/sub/grade)", "option"),
            ("New Value", "value"),
            ("Roll No", "roll")
        ], self.updFun)

    def updFun(self):
        opt = self.entries["option"].get().strip()
        val = self.entries["value"].get().strip()
        rNo = self.entries["roll"].get().strip()

        if opt not in ("name", "sub", "grade"):
            messagebox.showerror("Error", "Field must be one of name, sub or grade.")
            return
        if not (val and rNo):
            messagebox.showerror("Error", "Please provide both new value and roll
number.")
```

```
            return

        try:
            self.dbFun()
            query = f"UPDATE student SET {opt}=%s WHERE rollNo=%s"
            self.cur.execute(query, (val, rNo))
            self.con.commit()
            messagebox.showinfo("Success", f"Record Updated for Roll No {rNo}")
            self.showAll()
            self.popup.destroy()
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass

    # -------- Delete Record -------- #
    def delFrameFun(self):
        self.newWindow("Remove Student", [("Roll No", "roll")], self.delFun)

    def delFun(self):
        rNo = self.entries["roll"].get().strip()
        if not rNo:
            messagebox.showerror("Error", "Please enter Roll No.")
            return
        try:
            self.dbFun()
```

```
            self.cur.execute("DELETE FROM student WHERE rollNo=%s", (rNo,))
            self.con.commit()
            messagebox.showinfo("Success", f"Student Roll No {rNo} removed
successfully.")
            self.showAll()
            self.popup.destroy()
        except Exception as e:
            messagebox.showerror("Error", f"Error: {e}")
        finally:
            try:
                self.con.close()
            except:
                pass

    # -------- Database Connection -------- #
    def dbFun(self):
        # change 'yourpassword' to your actual DB password
        self.con = pymysql.connect(host="localhost", user="root",
passwd="Shra@2807", database="rec")
        self.cur = self.con.cursor()

    # -------- Reusable Small Window -------- #
    def newWindow(self, title, fields, command, combo=False):
        # Destroys any previous popup reference (safe)
        if hasattr(self, "popup") and self.popup.winfo_exists():
            try:
                self.popup.destroy()
            except:
                pass
```

```
        self.popup = tk.Toplevel(self.root)
        self.popup.title(title)
        self.popup.geometry("460x420+400+200")
        self.popup.configure(bg="#caf0f8")
        self.entries = {}

        tk.Label(self.popup, text=title, font=("Segoe UI Black", 20), bg="#00b4d8",
fg="white", relief="flat").pack(fill="x", pady=5)

        frame = tk.Frame(self.popup, bg="#caf0f8")
        frame.pack(pady=20)

        for i, (label, key) in enumerate(fields):
            tk.Label(frame, text=f"{label}:", font=("Segoe UI Semibold", 13),
bg="#caf0f8").grid(row=i, column=0, pady=10, padx=10)
            if combo and key == "option":
                # create combobox for selecting column
                self.option = ttk.Combobox(frame, values=("rollNo", "name", "sub"),
font=("Segoe UI", 12), state="readonly")
                self.option.set("Select Field")
                self.option.grid(row=i, column=1, padx=10, pady=10)
            elif combo and key == "value":
                # for combo mode we also need a separate entry for the value
                self.value = tk.Entry(frame, font=("Segoe UI", 12), width=20, bd=2,
relief="groove")
                self.value.grid(row=i, column=1, padx=10, pady=10)
            else:
                entry = tk.Entry(frame, font=("Segoe UI", 12), width=20, bd=2,
```

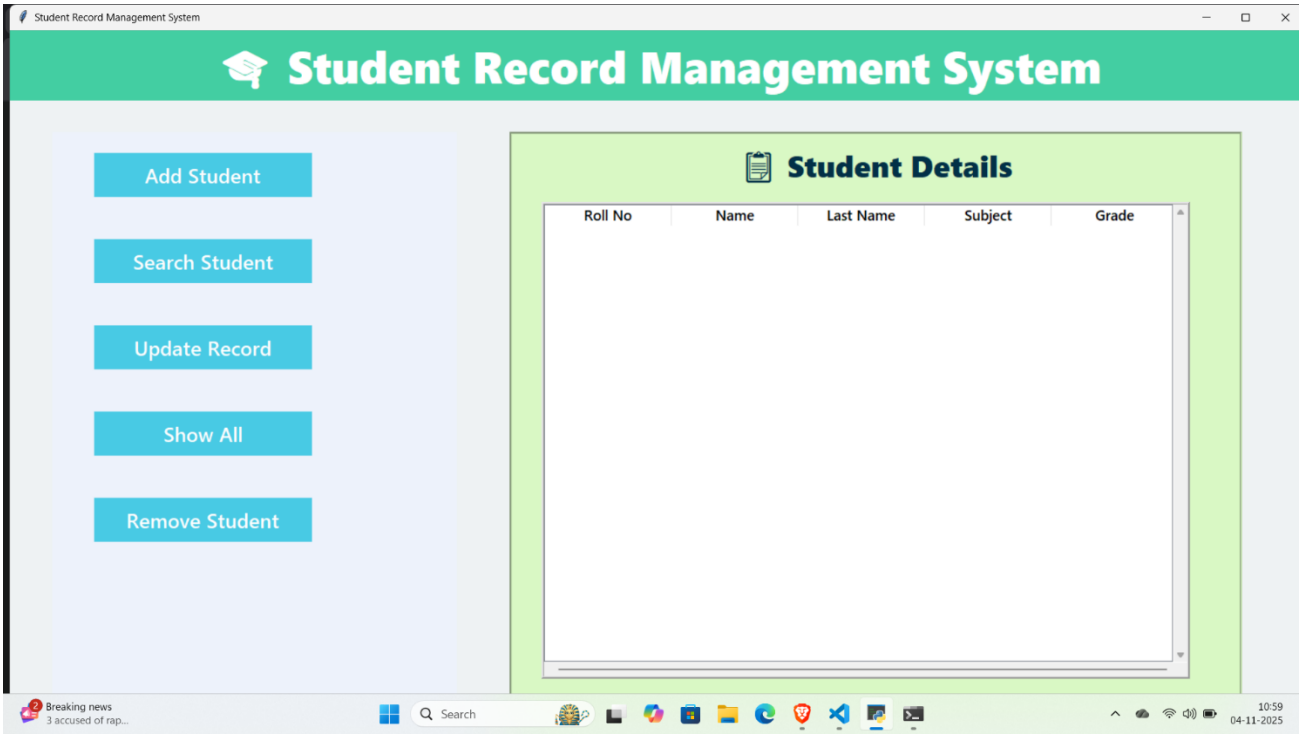
```
relief="groove")
            entry.grid(row=i, column=1, padx=10, pady=10)
            # store in entries dict for normal (non-combo) or update windows
            self.entries[key] = entry

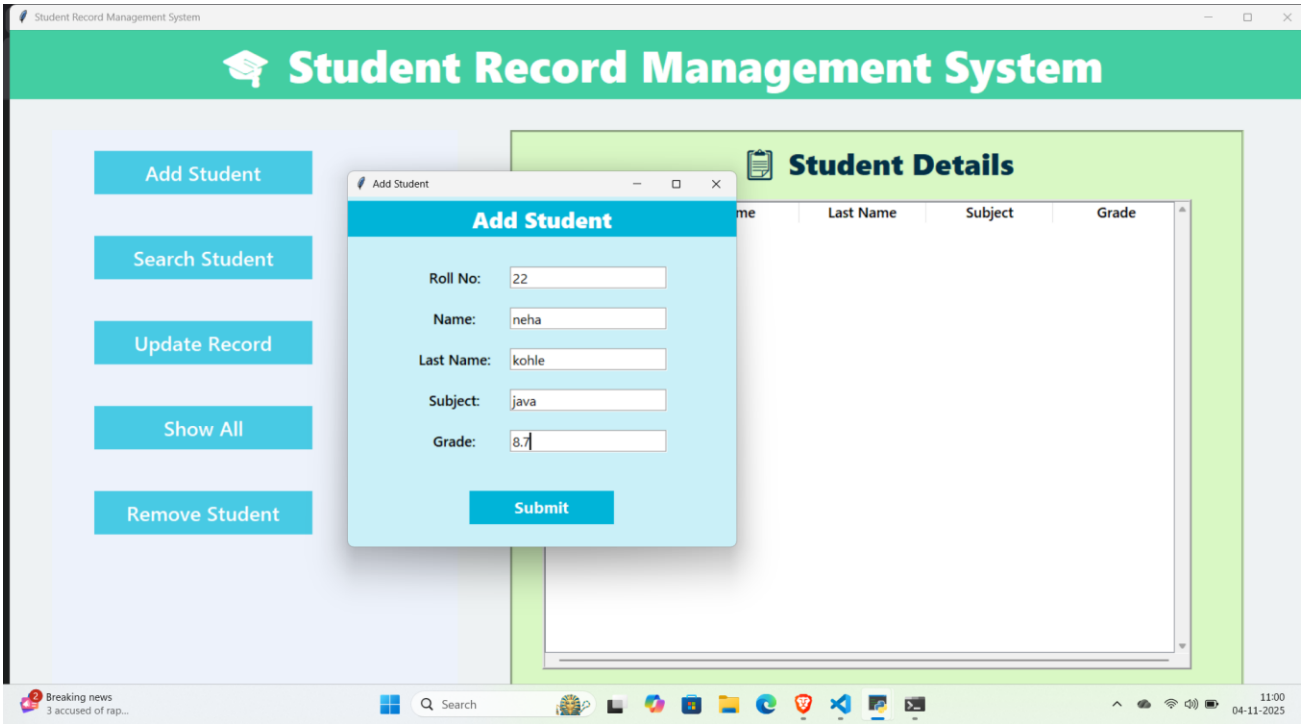
      tk.Button(self.popup, text="Submit", command=command, bg="#00b4d8",
fg="white",
              font=("Segoe UI", 14, "bold"), bd=0, relief="flat", width=15,
cursor="hand2").pack(pady=15)

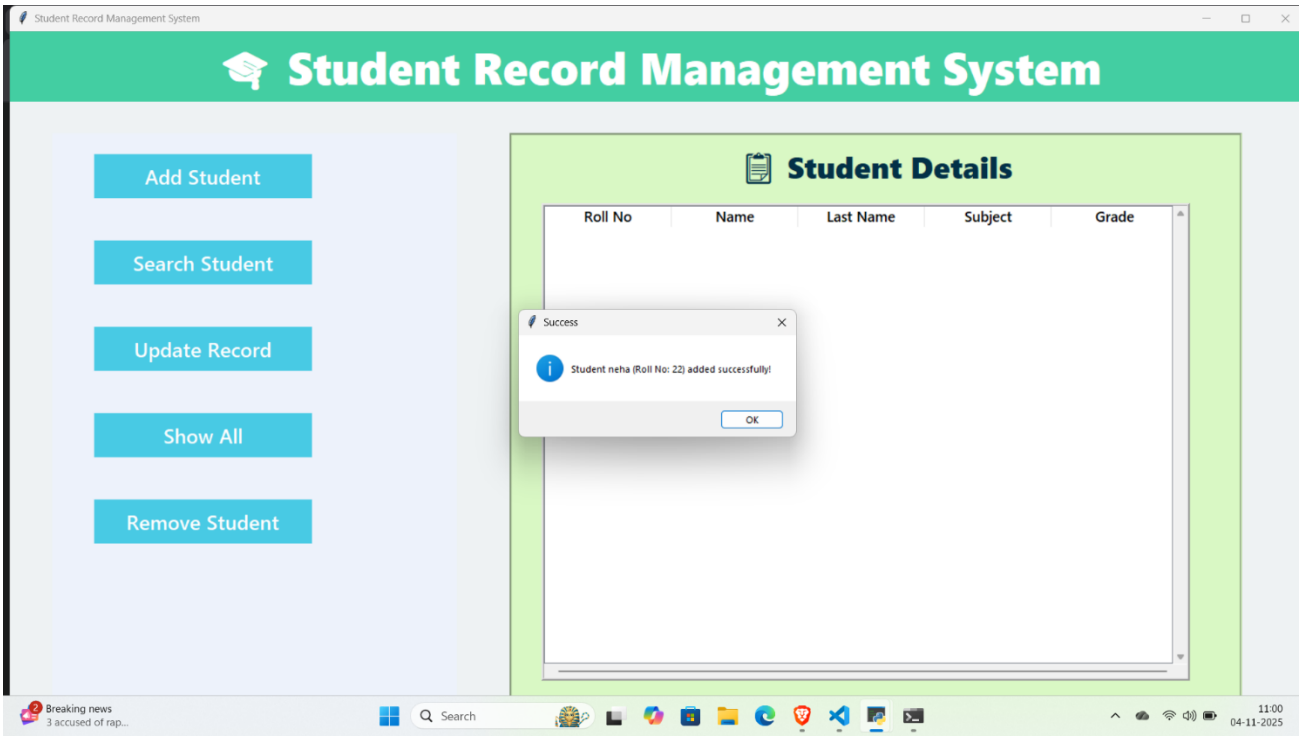
   def showAll(self):
      try:
         self.dbFun()
         self.cur.execute("SELECT * FROM student")
         data = self.cur.fetchall()
         self.table.delete(*self.table.get_children())
         for i in data:
            self.table.insert(', tk.END, values=i)
      except Exception as e:
         messagebox.showerror("Error", f"Error: {e}")
      finally:
         try:
            self.con.close()
         except:
            pass

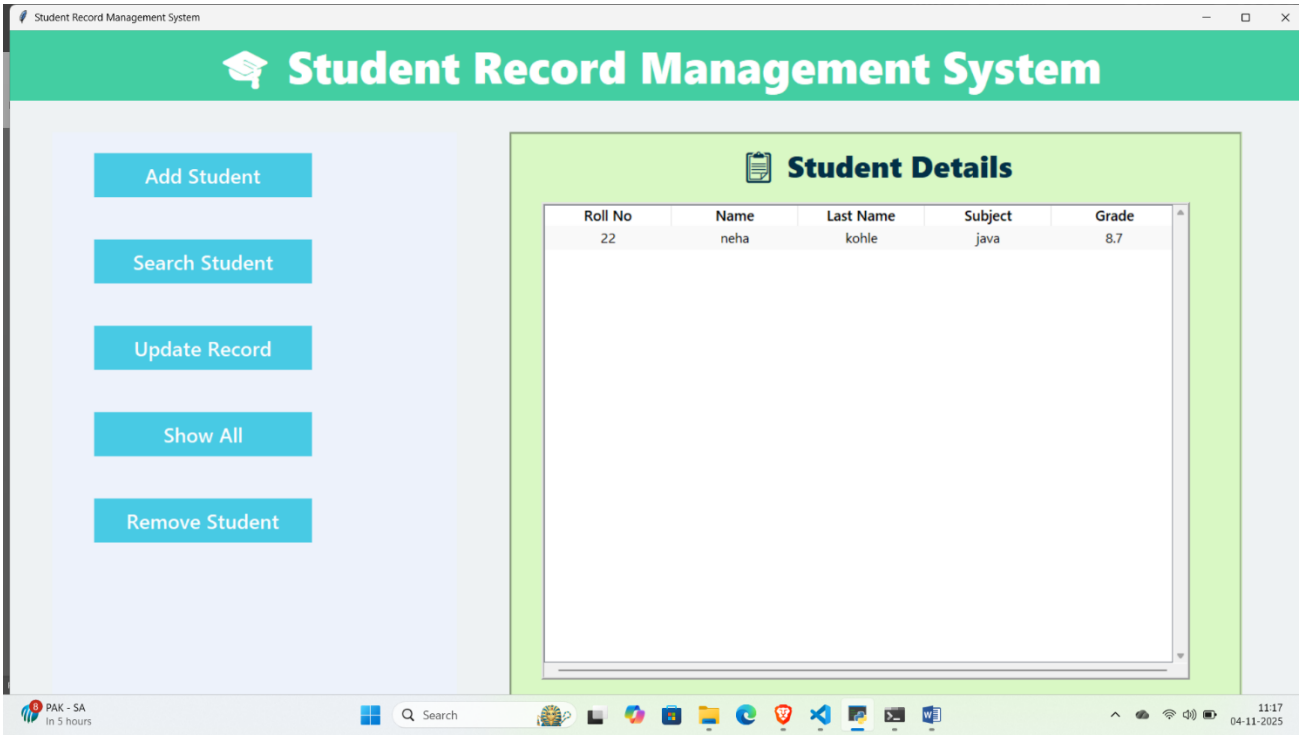
root = tk.Tk()
obj = StudentApp(root)
root.mainloop()
```

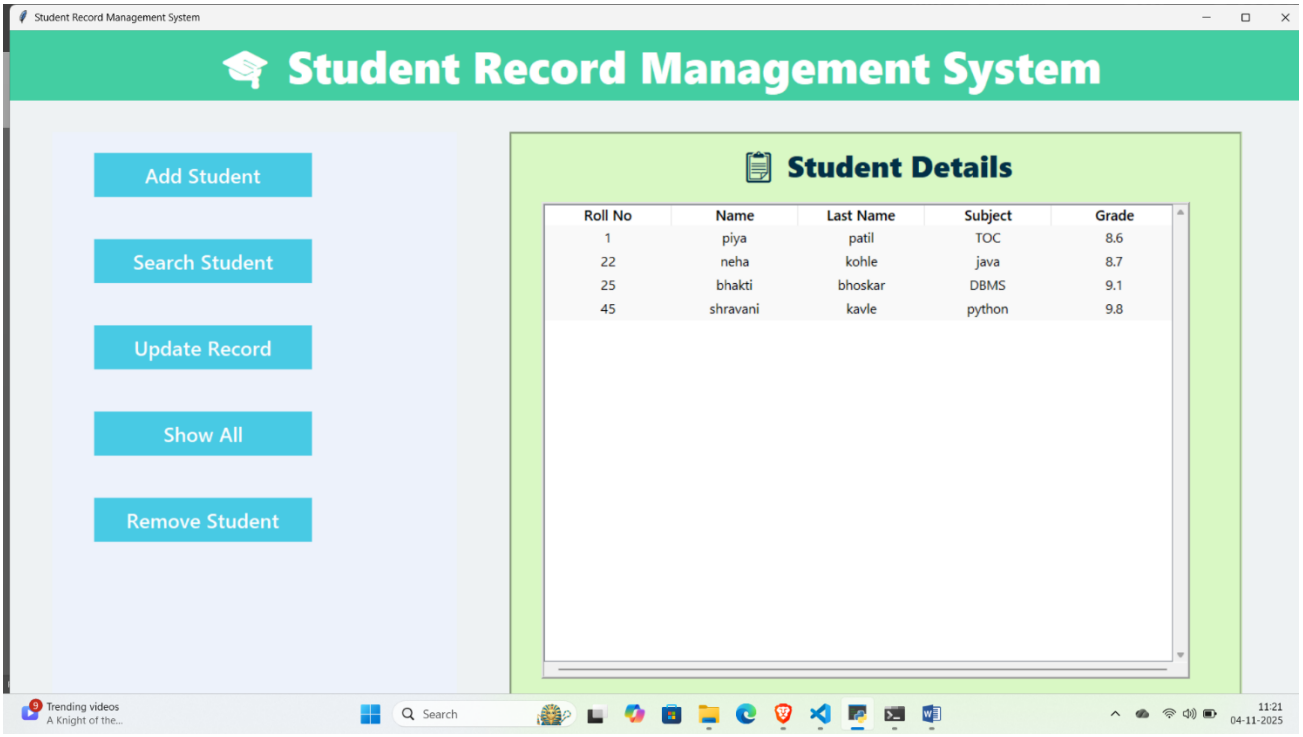
- **OUTPUT:**

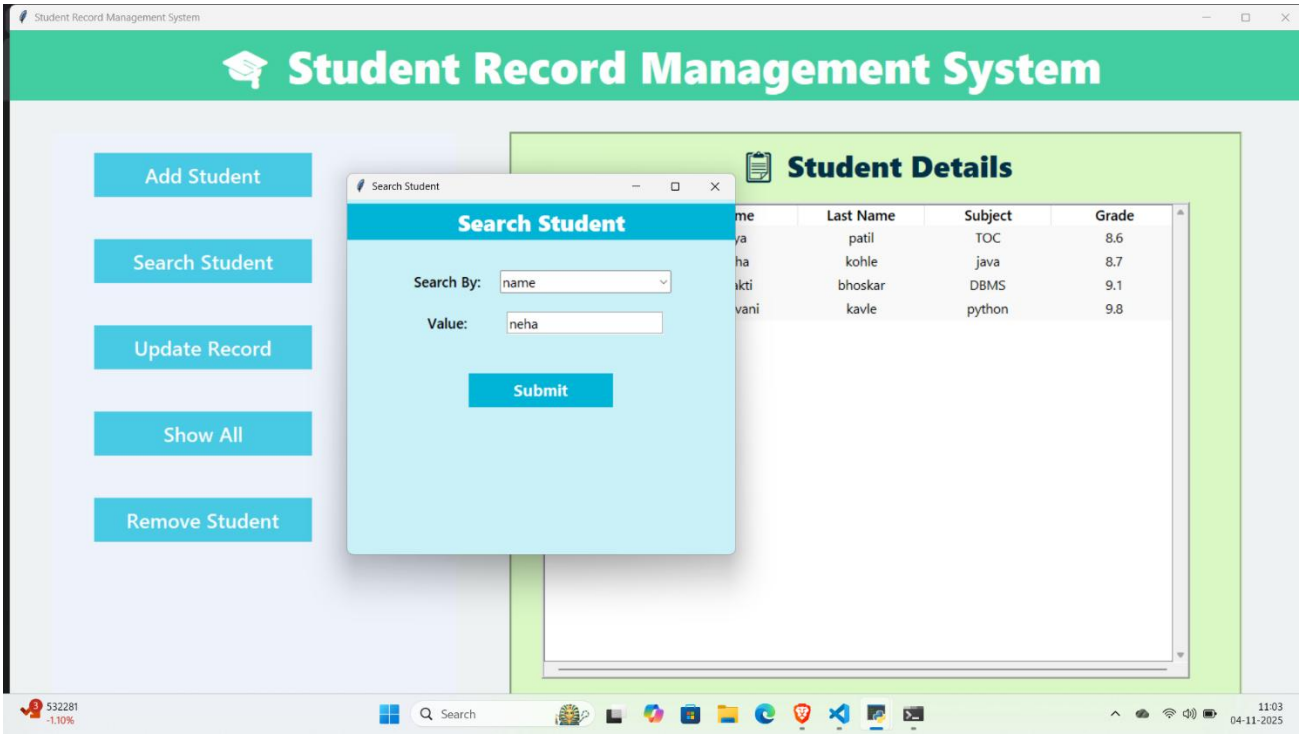

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade

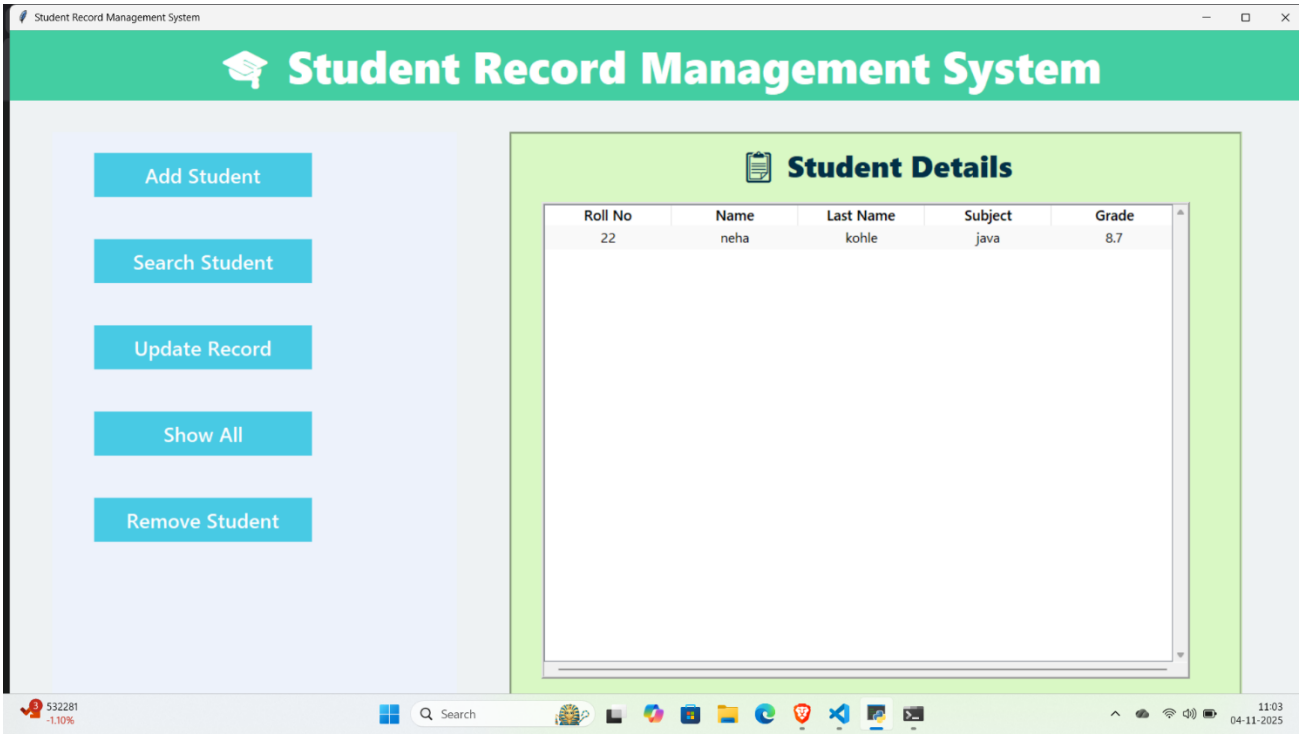

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Last Name
Subject
Grade
Add Student
Add Student
Roll No: 22
Name: neha
Last Name: kohle
Subject: java
Grade: 8.7
Submit

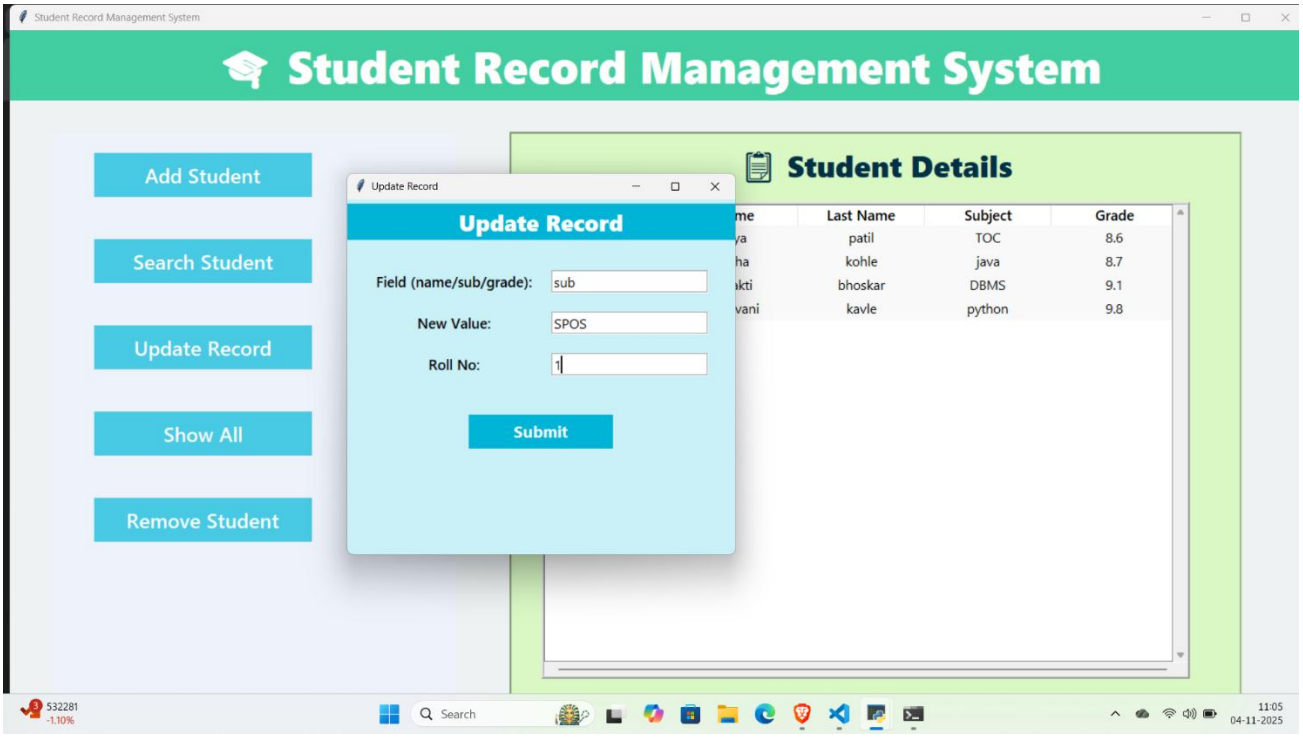

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
Success
Student neha (Roll No: 22) added successfully!
OK
Breaking news
3 accused of rap...
Search
11:00
04-11-2025

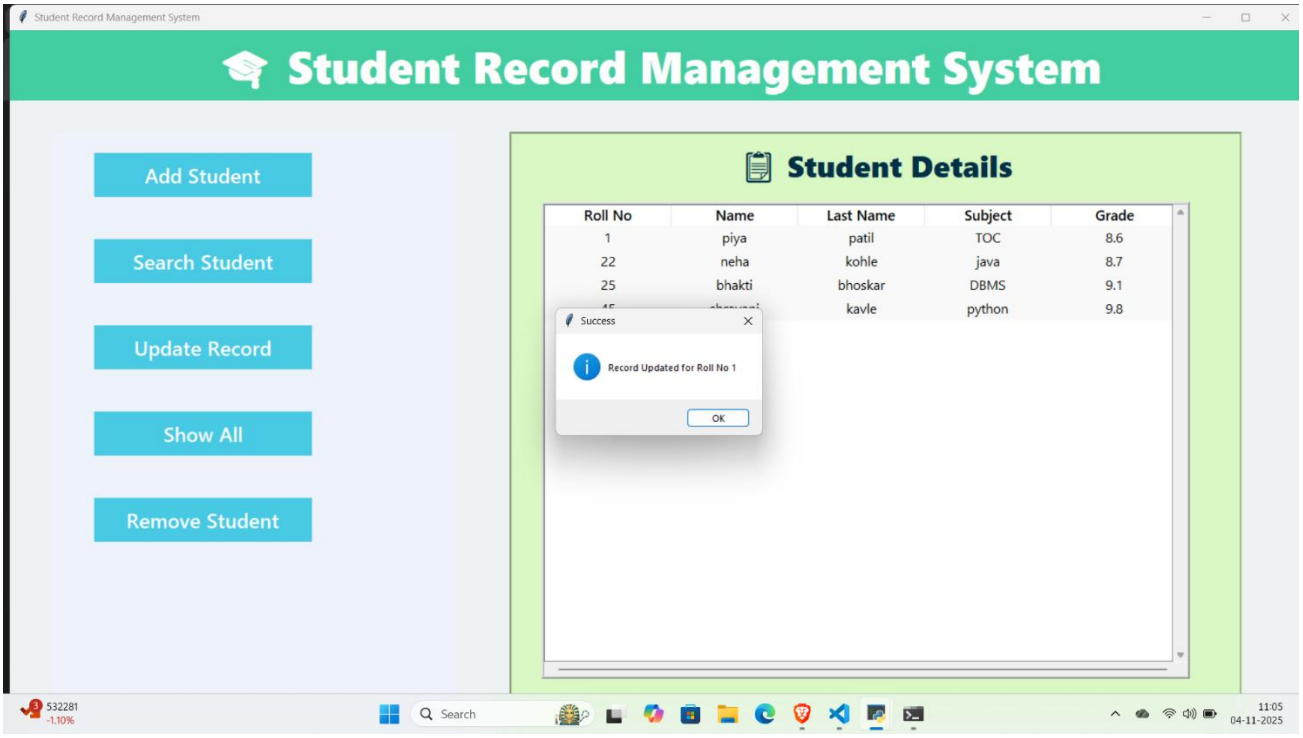

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
22
neha
kohle
java
8.7
PAK - SA
In 5 hours
Search
11:17
04-11-2025

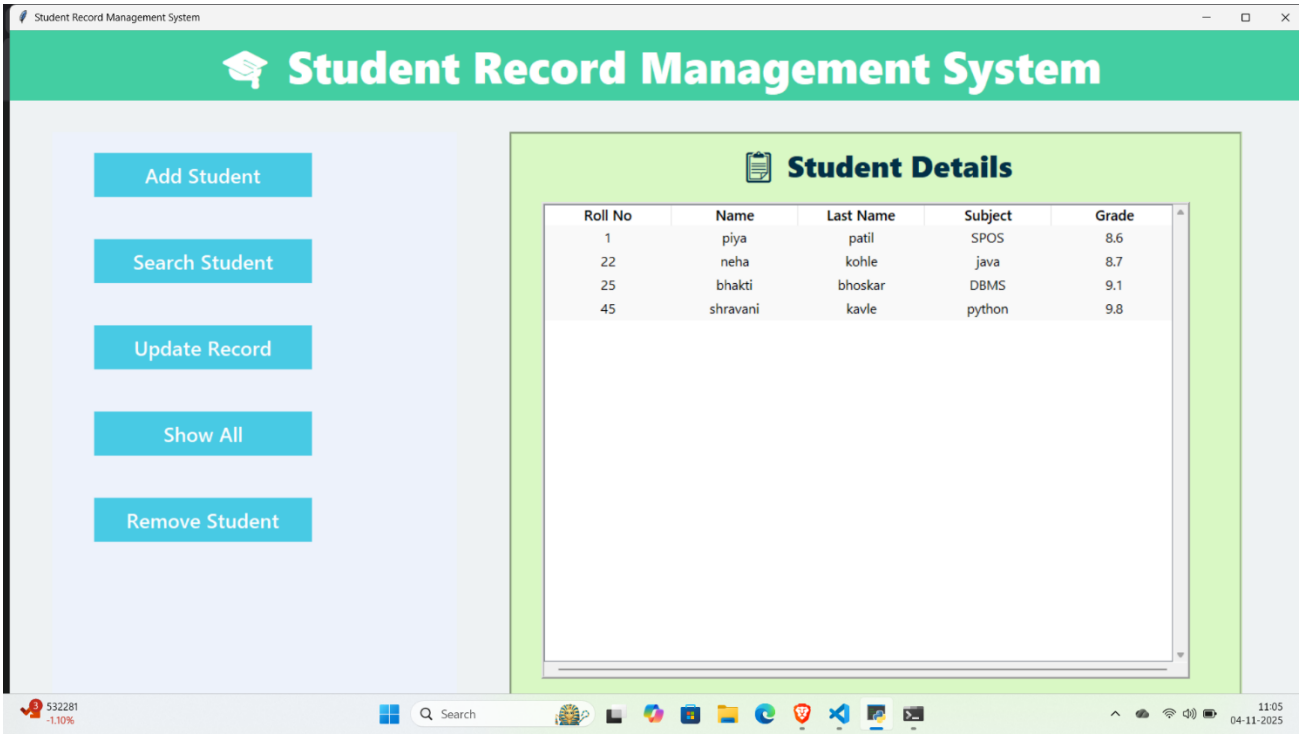

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
1
piya
patil
TOC
8.6
22
neha
kohle
java
8.7
25
bhakti
bhoskar
DBMS
9.1
45
shravani
kavle
python
9.8
Search
11:21
04-11-2025

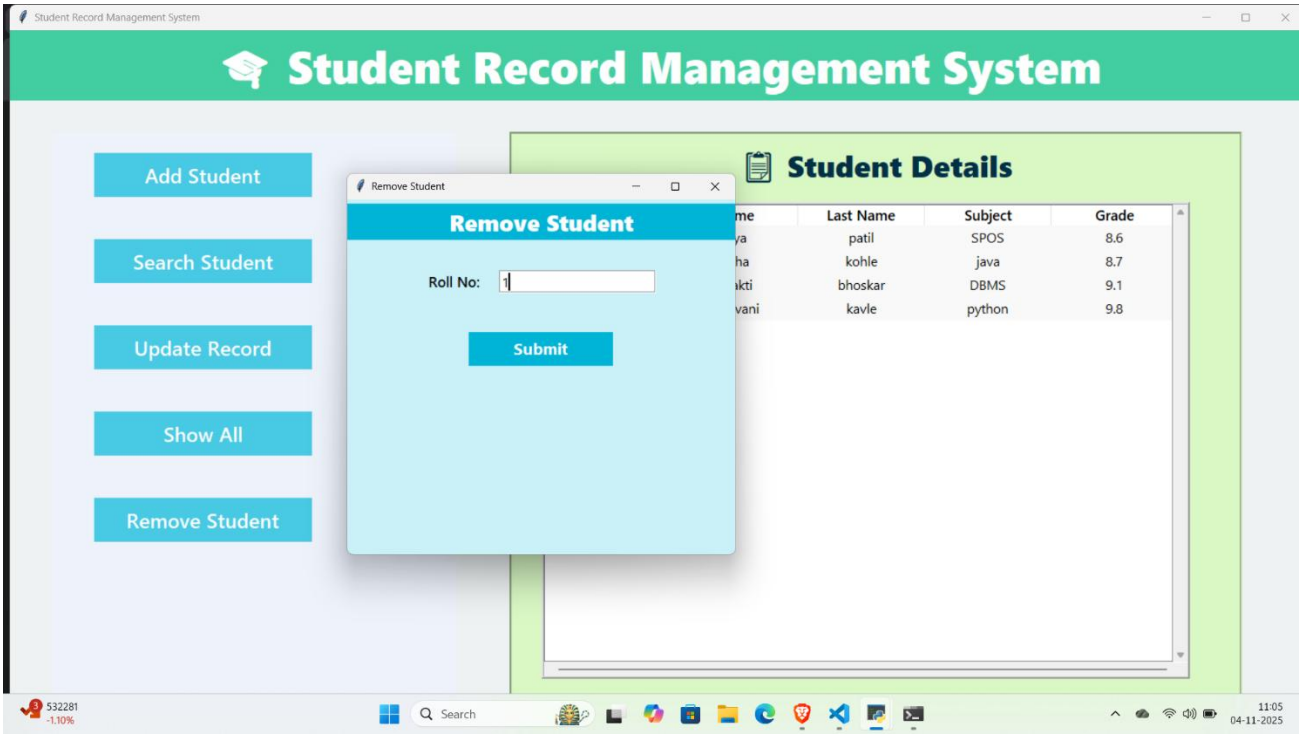

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Search Student
Search Student
Search By:
name
Value:
neha
Submit
Student Details
Last Name
Subject
Grade
patil
TOC
8.6
kohle
java
8.7
bhoskar
DBMS
9.1
kavle
python
9.8
Search
11:03
04-11-2025

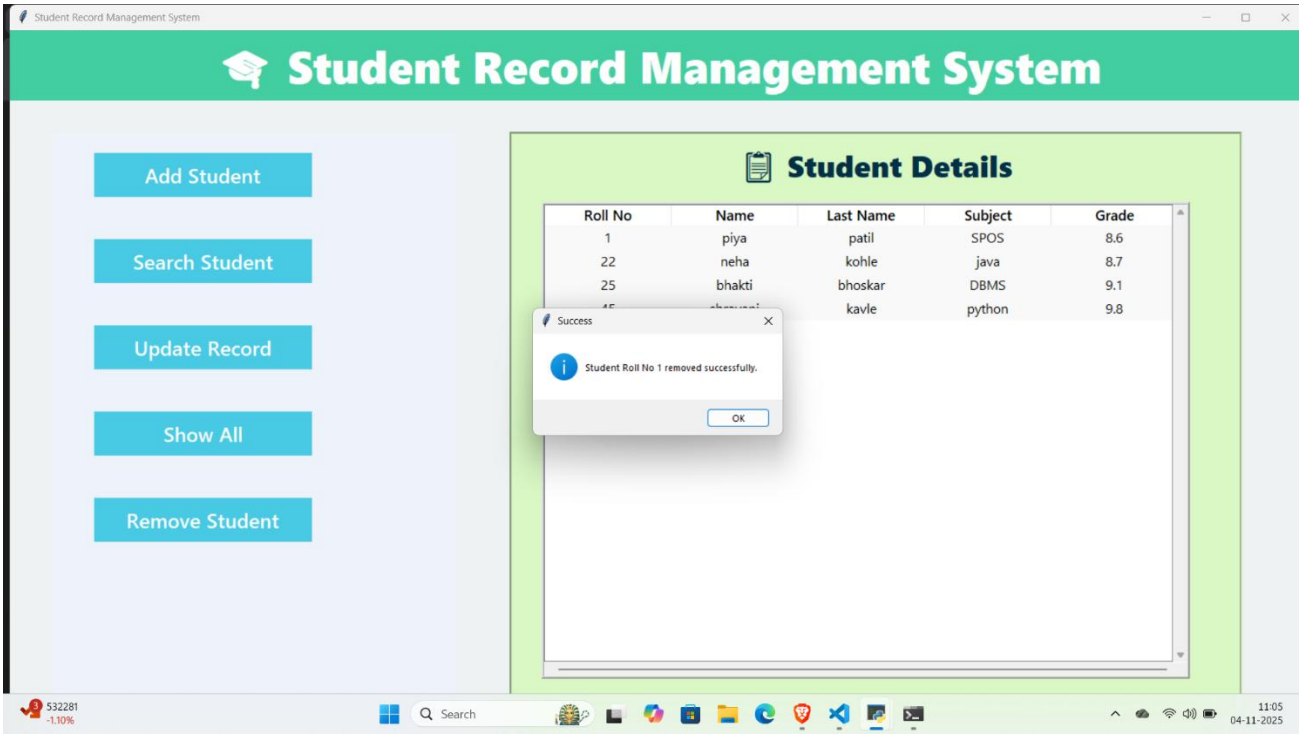

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
22
neha
kohle
java
8.7

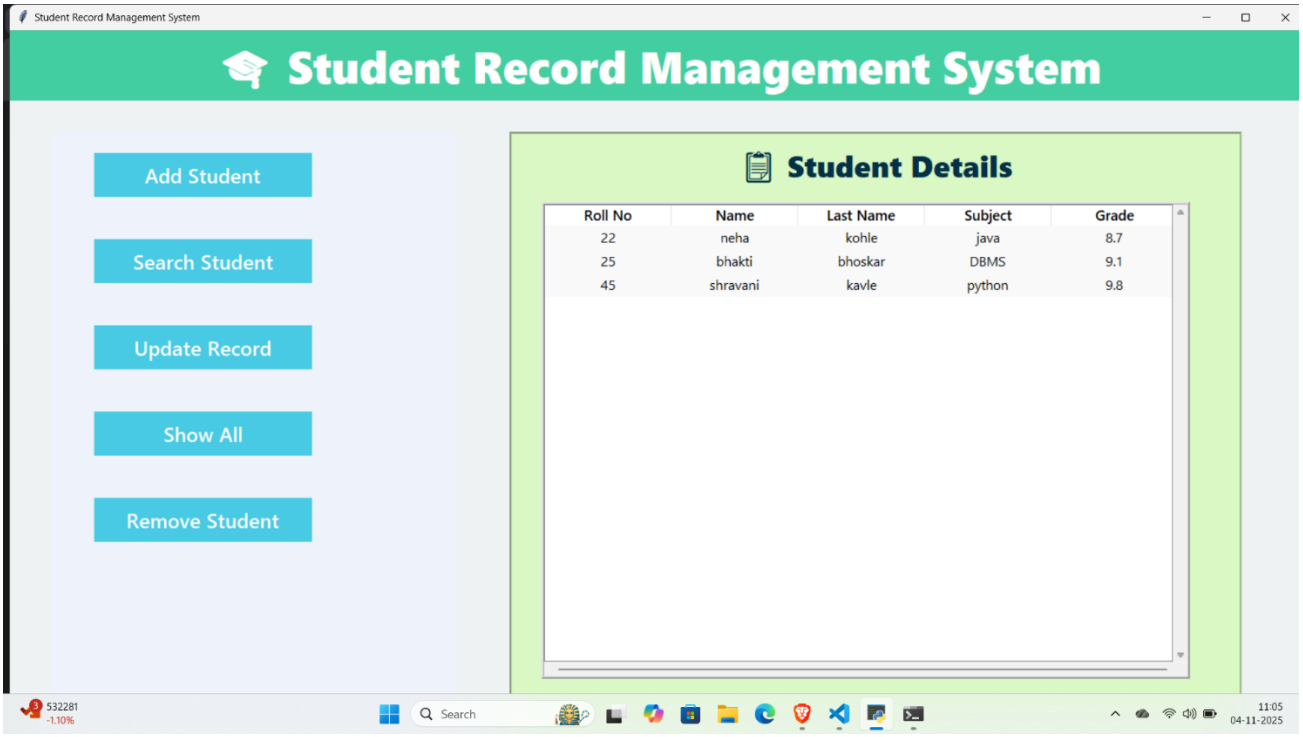

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Update Record
Update Record
Field (name/sub/grade):
sub
New Value:
SPOS
Roll No:
1
Submit
Student Details
Last Name
Subject
Grade
patil
TOC
8.6
kohle
java
8.7
bhoskar
DBMS
9.1
kavle
python
9.8


Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
1
piya
patil
TOC
8.6
22
neha
kohle
java
8.7
25
bhakti
bhoskar
DBMS
9.1
kavle
python
9.8
Success
Record Updated for Roll No 1
OK
Search
11:05
04-11-2025

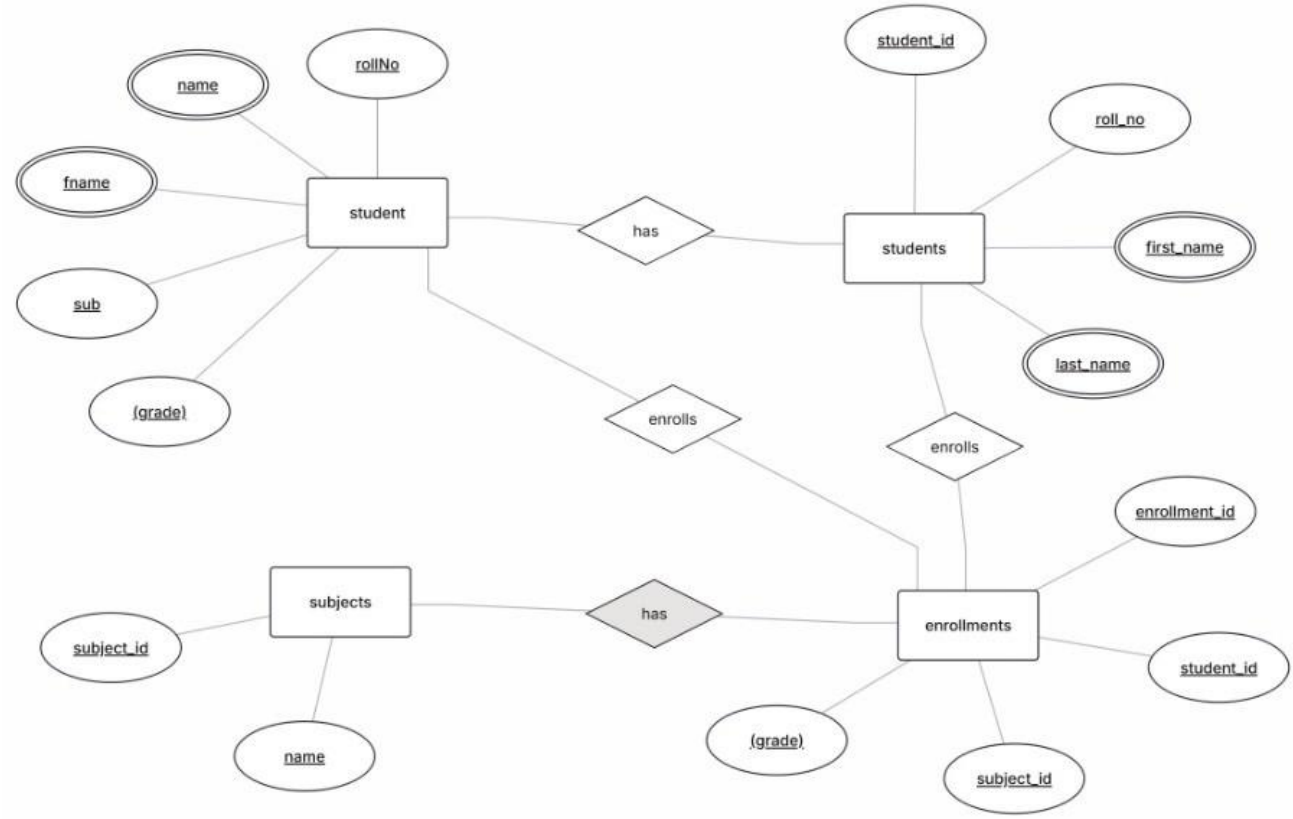

Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
1
piya
patil
SPOS
8.6
22
neha
kohle
java
8.7
25
bhakti
bhoskar
DBMS
9.1
45
shravani
kavle
python
9.8
Search
11:05
04-11-2025


Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Remove Student
Remove Student
Roll No:
1
Submit
Student Details
Last Name
Subject
Grade
patil
SPOS
8.6
kohle
java
8.7
bhoskar
DBMS
9.1
kavle
python
9.8
Search
11:05
04-11-2025


Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
1
piya
patil
SPOS
8.6
22
neha
kohle
java
8.7
25
bhakti
bhoskar
DBMS
9.1
kavle
python
9.8
Success
Student Roll No 1 removed successfully.
OK
Search
11:05
04-11-2025


Student Record Management System
Student Record Management System
Add Student
Search Student
Update Record
Show All
Remove Student
Student Details
Roll No
Name
Last Name
Subject
Grade
22
neha
kohle
java
8.7
25
bhakti
bhoskar
DBMS
9.1
45
shravani
kavle
python
9.8
532281
-1.10%
Search
11:05
04-11-2025

- **ER DIAGRAM :**


student_id
rollNo
name
roll_no
fname
student
has
students
first_name
sub
last_name
(grade)
enrolls
enrolls
enrollment_id
subjects
has
enrollments
subject_id
student_id
name
(grade)
subject_id

- **RELATIONSHIP OF TABLES :**

- students → enrollments : One student can have many enrollments → One-to-Many relationship
- enrollments.student_id → FK → students.student_id
- subjects → enrollments : One subject can have many enrollments → One-to-Many relationship
- enrollments.subject_id → FK → subjects.subject_id
- students ↔ subjects : A student can enroll in multiple subjects, and a subject can have multiple students → Many-to-Many relationship (through enrollments table)

- **CONCLUSION :**

The Student Record Management System simplifies the process of managing student data by providing an efficient, secure, and organized digital solution. It enhances accuracy, saves time, and supports smooth administrative operations within educational institutions.